EE2016 Microprocessor Theory and Lab (Aug-Nov 2025)
ARM assembly language programming based Tutorial.

1) For questions (i) to (v) give the values of the registers/flags post the execution of the given instruction.

(i) PRE r5 = 5, r7 = 8
MOV r7, r5, LSL #3 ;
POST r5 = 5, r7 = 40

(ii) PRE cpsr = nzcvqiFt_USER
r0 = 0x00000000
r1 = 0x80000004
MOVS r0, r1, LSL #1
POST cpsr = , r0 =

(iii) PRE
r0 = 0x00000000
r1 = 0x00000000
r2 = 0xf0000003
r3 = 0x00000008
UMULL r0, r1, r2, r3 ;
Post : r0 =, r1=

iv) PRE
r0 = 0x00000000
r1 = 0x80000000

mem32[0x00009000] = 0x01010111
mem32[0x00009004] = 0x01234567

What will the values of the relevant registers post the execution of each of the following instructions (Note: 4(a), 4(b) and 4(c) are separate questions).

a) LDR r0, [r1, #4]
b) LDR r0, [r1, #4]!
c) LDR r0, [r1], #4

v) PRE
r0 = 0x00009000
r1 = 0x00000003
r2 = 0x00000004
r3 = 0x00000005
STMIB r0!, {r1-r3}
LDMDA r0!, {r4-r6}
Post : Find contents of registers r0 to r6.

2) For the below ARM assembly code, trace the values as it executes that will be placed into the registers

R4, R5, and R6.

```
        MOV R4, #5
        MOV R5, #4
        MOV R6, #3
again   MOV R7, R4
        ADD R4, R5, R4
        MOV R5, R7
        SUBS R6, R6, #1
        BNE again
```

R4: 5 9 e 23
R5: 4 5 9 e
R6: 3 2 1 0
R7: 5 9 e

3) Translate the below C fragment into an equivalent ARM assembly language program, using registers corresponding to the variable names. (As it happens, this code has the effect of placing into r2 the product of r0 and r1's initial values.)

```
int r0 = 4;
int r1= 3
int r2 = 0;
while (r1 != 0) {
    if ((r1 & 1) != 0) {
        r2 += r0;
    }
    r0 <<= 1;
    r1 >>= 1;
}
while (1); // halting loop
```

```
mov R1 #0x00..3
 "  R2
 "  R0
 B here
AND R3, R1, #0x...1
CMP R2, #0x0..0
BEQ 0x00..20C
ADD R2, R2, R0
mov R0, R0, LSL #1
mov R0, R1, ASR #1
here CMP R1, #0x...0
BNE 0x000..1FC
STOP B STOP
```

4) For the ARM assembly code given below, find the values of the registers R0, R1, and R2 just before the program gets into the infinite loop.

```
 ADD R0, PC, #nums
loop    LDRB R1, [R0], #1
        SUBS R2, R1, #3
        BLE loop
halt    B halt
```

R0 -> address of nums
-> load byte from R0 to R1, then do R0+1
-> R2 = R1 - 3
Branch if R1 - 3 ≤ 0

R0 = A
R1 = 1
R0 = A+1
R2 = -2

```
nums    DCB 1, 2, 3, 4, 5
```

5) Write a program in ARM assembly to count the number of 1s in a 32-bit number stored in r1. Save the result in r3.

6) Convert the following C program to a program to ARM assembly. Assume that the base address of the array is stored in r0

```
void addNumbers(int a[100]) {
        int idx;
        int sum = 0;
        for (idx = 0; idx < 100; idx++){
                sum = sum + a[idx];
        }
}
```

7) For the ARM assembly code given below, trace the values as it executes that will be placed into the registers R0, R1, and R2.

```
        MOV     R0, #48
        MOV     R1, #5
        MVN     R2, #0    ; MVN, not MOV
loop1   ADD     R2, R2, #1
        CMP     R0, R1, LSL R2
        BGT     loop1
        SUBLTS  R2, R2, #1
        BLT     done
loop2   CMP     R0, R1, LSL R2
        SUBGE   R0, R0, R1, LSL R2
        SUBS    R2, R2, #1
        BGE     loop2
Done
```

8) For the ARM assembly code given below, trace the values as it executes that will be placed into the registers R1, R2, and R3. Note that the first line places the number 32 into R0.

```
      ADD R0, PC, #nums ; places the address of nums[0] == 1 into R0
      MOV R1, #0
loop  ADD R2, R0, R1, LSL #2
      LDR R3, [R2]
      ADD R1, R1, R3
      CMP R1, #10
      BLT loop
done  B done
nums  DCD 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89
```

→ $R_0 + R_1 \times 2^2$

0 1 2 3 4 5 6 7 8 9 10